The Journal of Open Source Software

PySlyde: A Lightweight, Open-Source Toolkit for Pathology Preprocessing

Chima Eke[1*], Anthony Baptista[1,2*], Holly Rafique[1*], Elizabeth Ing-Simmons[1,3*], Enrico Parisini[1,3], Mengyuan Li[1,3], Fathima Mohamed[1], Ananya Bhalla[1,5], Lucy Ryan[1], Michael Pitcher[1,3], Concetta Piazzese[3,6], Fan Zhang[3,7], James Graham[3,4], Dinis Pedro Calado[5], Christopher R. S. Banerji[1,2], Anita Grigoriadis[1,3¶], and Gregory Verghese[1,3¶]

1 Cancer Bioinformatics, School of Cancer and Pharmaceutical Sciences, Faculty of Life Sciences and Medicine, King's College London, London, UK **2** The Alan Turing Institute, The British Library, London, UK **3** PharosAI, London, UK **4** eResearch, King's College London, London, UK **5** The Francis Crick Institute, London, UK **6** Barts Life Sciences, Barts Health NHS Trust, London, UK **7** University of Cambridge, Cambridge, UK ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Julia Romanowska

Reviewers:

- @manunicholasjacob
- @showdawn
- @moerlemans

Submitted: 09 March 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Artificial intelligence (AI) represents a transformative opportunity for precision medicine, with the potential to radically improve patient outcomes (Song et al., 2023; Verghese, Lennerz, et al., 2023). Advances in computational pathology are enabling the analysis of complex histopathological data at unprecedented scale and depth. By combining modern computer vision, machine learning, and pathology, researchers can predict quantitative, molecular (Dey et al., 2025) and digital biomarkers, detect morphological and tissue patterns (S. Chen et al., 2026), and develop predictive models for diagnostics, prognosis (Verghese, Li, et al., 2023) and treatment response to complement human expertise (Laak et al., 2021).

Whole slide images (WSIs), resulting from the digitisation of histopathology slides, offer valuable spatial and morphological insights that can deepen our understanding of cancer biology. However, their size (gigapixel-scale), complexity, and variability introduce significant computational and standardisation challenges for downstream tasks (McGenity et al., 2024). Preprocessing WSIs, including by tissue detection, tessellation (splitting into smaller tiled images), stain normalisation (Macenko et al., 2009; Vahadane et al., 2016) and annotation parsing are essential steps before any downstream AI analysis can occur (Pocock et al., 2022). Building robust, and reproducible pipelines for WSI preprocessing is fundamental to computational pathology research. However, existing workflows often rely on ad hoc proprietary scripts, fragmented tools, and inconsistent formats, making standardisation and reproducibility difficult.

PySlyde is a lightweight, open-source Python toolkit built on top of OpenSlide (Goode et al., 2013), designed to address this gap and provide an intuitive approach to quickly preprocess WSIs. It provides a simple API to perform WSI loading, annotation handling, tissue detection, tile generation, and feature extraction with support for the latest histopathology foundation models. PySlyde aims to lower the barrier to entry for computational pathology researchers, standardise preprocessing workflows, and accelerate the development of AI-ready datasets for computational pathology research, enabling researchers to focus on model development and other downstream tasks.

DRAFT

The Journal of Open Source Software

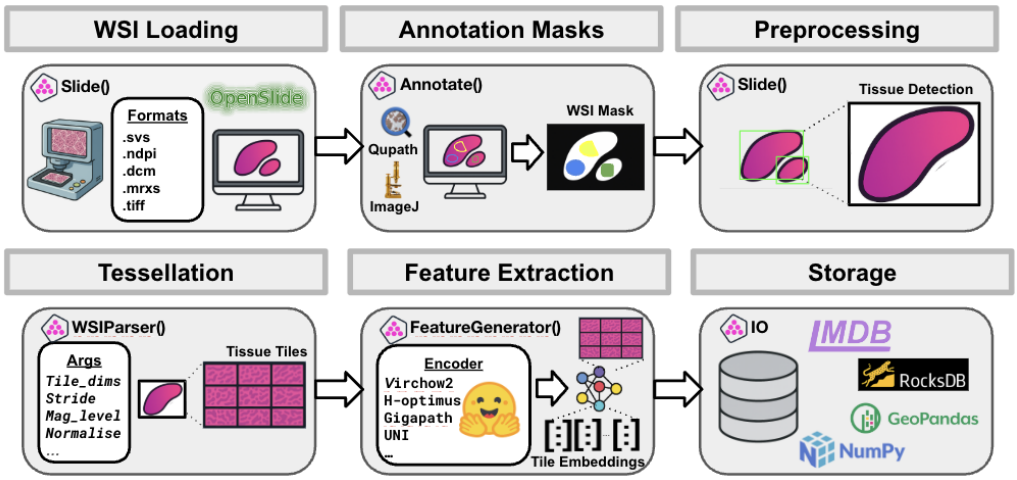


Figure 1: Schematic overview of the WSI processing workflow supported by PySlyde, illustrating the core functionalities across the main classes and modules `Slide`, `WSIParser`, `FeatureGenerator` and `IO`. Top row, from left to right: (i) WSI loading using compatible formats through OpenSlide; (ii) annotation mask generation with support for QuPath or ImageJ formats; and (iii) preprocessing steps such as tissue detection. Bottom row, from left to right: (iv) tessellation of WSIs into tissue tiles using the `WSIParser` class; (v) feature extraction from tiles with support for pretrained encoders such as Virchow2, H-optimus, Gigapath, or UNI; and (vi) structured storage of tile embeddings and metadata using RocksDB (via Rocksdict), NumPy, or LMDB.

Statement of Need

WSI preprocessing is a foundational but time-consuming step in computational pathology. Common tasks, such as annotation parsing, binary and annotation mask generation, stain normalisation, and tessellation often involve custom scripts, multiple packages and ad hoc pipelines built for a single dataset or project. This fragmentation hinders reproducibility, increases technical debt, and slows the development and translation of AI methods into clinical workflows. The lack of standardised preprocessing practices thus remains one of the key barriers preventing the seamless integration of AI into diagnostic pathology (Bilal et al., 2025).

While powerful frameworks exist for model training and inference, with others providing distinct sections of the pipeline, few provide a simple, consistent interface for preparing WSIs at scale. In particular there is limited capability to integrate contributions from clinicians (such as pathologist annotations) into the computational pathology research pipeline. PySlyde directly addresses this unmet need by providing a modular, well-documented, and extensible preprocessing toolkit that integrates seamlessly with existing ecosystems, including OpenSlide, PyTorch, and modern foundation models enabling researchers to build reproducible, scalable pipelines with minimal code.

State of the Field

Several open-source tools support WSI analysis in computational pathology. At the foundational level, OpenSlide (Goode et al., 2013) provides low-level image access and metadata retrieval across vendor-specific formats, underpinning higher-level preprocessing libraries. TIAToolbox (Pocock et al., 2022) provides a broad collection of tissue image analysis tools, including WSI reading, tissue masking, patch extraction, stain normalisation, model inference and visualisation. It supports annotation storage and GeoJSON import, and foundation model feature extraction. PathML (Omar et al., 2025; Rosenthal et al., 2022) supports diverse pathology imaging formats and modular preprocessing pipelines, including tissue detection, stain normalisation, tile masks,

DRAFT

metadata and parallel processing. Slideflow (Dolezal et al., 2024) combines WSI preprocessing with model training, multiple-instance learning and feature extraction using seven documented pathology-specific pretrained encoders. PySlyde provides capability for additional encoders to be incorporated as the state of the art evolves. Specialised tools address distinct stages of this workflow: TRIDENT (Zhang et al., 2025) focuses on large-scale tissue segmentation, patch extraction and foundation-model embeddings, while HistoQC (Janowczyk et al., 2019) provides tissue quality assessment, artefact masks and cohort-level quality review. In contrast, PySlyde is designed specifically as a modular preprocessing layer, focusing on efficient data preparation for downstream machine learning workflows.

PySlyde's contribution is a focused, modular workflow combining annotation-aware preprocessing, configurable tiling, feature extraction and storage. It supports annotations from QuPath (Bankhead et al., 2017) and ImageJ (Schneider et al., 2012), together with externally generated masks from tools such as HistoQC, allowing labelled regions to guide mask generation and tile selection. Slide-level spatial coordinates enable interoperability with geospatial libraries such as GeoPandas and Shapely, facilitating spatial analysis workflows that extend beyond traditional image processing pipelines. Extensible encoder interfaces incorporate pathology foundation models such as Virchow2 (Zimmermann et al., 2024), H-optimus (Saillard et al., 2024), and UNI (R. J. Chen et al., 2024), while file outputs, LMDB (Symas Corporation, n.d.) and RocksDB (Meta Platforms, Inc., n.d.) accommodate different downstream access patterns. PySlyde addresses a specific gap in the current ecosystem: providing a minimal, modular preprocessing backbone that integrates complex histopathology, WSI annotation handling, geospatial representations, foundation model feature extraction and efficient storage within a single lightweight framework.

Software Design

Architecture

PySlyde is a lightweight and modular WSI preprocessing framework that separates slide access, annotation handling, spatial sampling, encoder inference and persistence (Figure 1). Rather than providing a tightly coupled end-to-end pipeline, PySlyde enables researchers to integrate specific preprocessing stages into existing machine learning workflows.

The `Slide` class represents a single WSI with its annotations and metadata, providing spatial masking and region extraction. It inherits directly from OpenSlide, avoiding reimplementation of vendor-specific readers; format support depends on the installed OpenSlide version. The `Annotations` class parses supported annotation formats into labelled coordinate sequences for mask generation.

The `WSIParser` class manages spatial partitioning, tile selection and extraction. Separating sampling from slide representation allows a slide to be processed using different boundaries, tile dimensions, strides and pyramid levels, enabling gigapixel images to be processed efficiently under different region-selection and magnification strategies.

The `FeatureGenerator` class encapsulates model construction, checkpoint loading, image transforms and embedding generation behind a common encoder interface. `WSIParser` coordinates feature extraction through this interface, keeping model-specific operations outside the tiling logic.

The IO module provides persistence through file-based outputs and key–value stores, including LMDB (Symas Corporation, n.d.) and RocksDB (Meta Platforms, Inc., n.d.). Storage-specific operations are delegated to writer components, allowing extraction outputs to be directed to different backends.

DRAFT

Data Flow

A typical processing workflow begins with a WSI and optional annotations or externally generated label masks (Figure 1). Annotations can be rasterised into masks, while annotation boundaries, tissue detection, or user-specified regions define the processing area. The `pyslyde.masks` interface provides access to rasterised masks stored at full or reduced resolution through a common level-0 coordinate system. Within the selected area, `WSIParser` first generates and stores tile coordinates according to the sampling configuration. These locations can be filtered using mask-label coverage or reduced by random sampling. Alternatively, `WSIParser.filter_by_func` reads candidate tiles and applies a user-supplied rejection function, for example, to exclude tiles with entropy below a threshold or mean pixel intensity above a threshold. The parser retains the selected coordinates rather than caching the images.

Following selection, `WSIParser.extract_tiles` reads the retained image regions on demand at the configured resolution and yields each RGB tile with its coordinates. Optional preprocessing includes tile-level masking and stain normalisation. `WSIParser.extract_masks` separately yields the corresponding regions from a supplied mask. For feature extraction, `WSIParser.extract_features` reads each retained tile and passes it to `FeatureGenerator`, which applies encoder-specific image transforms, runs inference, and validates the resulting embedding. Each embedding is yielded with its source coordinates. These methods yield tiles, masks, and embeddings through separate generators with a shared coordinate–array output structure. Storage writers consume these pairs incrementally, saving images or arrays to the selected backend without first collecting all tiles or embeddings in memory. Optional embedding postprocessing is provided by `pyslyde.util.pca`, which supports incremental PCA fitting in batches from generated or stored embeddings. The utility returns a fitted PCA model, which the user can then apply to reduce embedding dimensionality and save the resulting vectors. `Stitching` enables users to reconstruct a processed slide region from saved image or mask patches using their spatial coordinates. This optional utility supports visual inspection, with configurable overlap handling and output resizing.

Design choices and trade-offs

Several architectural trade-offs guided this design. PySlyde prioritises modularity over a monolithic pipeline structure. While tightly integrated frameworks simplify end-to-end workflows, they constrain researchers to predefined processing stages. PySlyde separates spatial sampling, encoder inference and persistence to allow these stages to be configured independently. This flexibility requires consistent handling of coordinates and model-specific input requirements across component boundaries. PySlyde builds on OpenSlide (Goode et al., 2013) for WSI access. This reduces maintenance overhead and improves interoperability with existing pathology infrastructure. Annotation parsers convert supported inputs into a common representation of labelled coordinate sequences. This allows downstream mask generation and region selection to operate independently of the annotation source, enabling collaboration with a diverse clinical team; however, each format requires a maintained parser and explicit handling of its coordinate conventions, and geometry types. PySlyde supports both file-based outputs and key-value storage through LMDB and RocksDB, accommodating different downstream access patterns but introducing backend dependencies and maintenance requirements. The core installation has a small dependency set, deep-learning and storage backends (PyTorch, TensorFlow, RocksDB) are optional extras installed only when needed.

Research Impact Statement

PySlyde has been used in multiple ongoing computational pathology projects within the Cancer Bioinformatics Team at King's College London (London, UK), including multimodal outcome prediction studies in breast, head and neck and pan-cancer cohorts, with several manuscripts currently under review or in preparation. It is also used within the PharosAI

DRAFT

Research Ventures Catalyst programme for large-scale histopathology data curation across collaborating institutions including Guy's and St Thomas' NHS Foundation Trust, Queen Mary University of London, Barts Health NHS Trust, and King's College London. These applications, alongside a poster at the *ESMO AI Congress, Berlin 2025*, demonstrate its immediate research utility and support expectations for wider adoption within computational pathology workflows.

AI Usage Disclosure

AI-assisted development tools (e.g., Cursor (Anysphere, n.d.)) were used to support limited refactoring and code maintenance tasks. AI-assisted changes were reviewed through standard code review and Continuous Integration/Continuous Delivery (CI/CD) validation processes on GitHub to ensure correctness and reliability. OpenAI Codex (OpenAI, n.d.) was used to assist with generating and debugging code for the core mathematical operations of the Macenko and Vahadane stain normalisation methods. The authors reviewed, modified, and tested the generated code and take responsibility for the final implementation and results. Generative AI tools were also used to support the writing, reviewing and editing of this manuscript, with all final content verified and edited by the authors.

Maintenance and Availability

PySlyde is actively maintained by contributors from PharosAI, King's College London, The Alan Turing Institute, The Francis Crick Institute and Barts Health NHS Trust. The project is hosted on GitHub under an MIT license and follows open development practices, including community issue tracking and pull request review. Continuous integration is implemented via GitHub Actions, with automated testing across supported Python versions to ensure reliability and stability. The codebase includes unit tests and example workflows to support reproducibility and long-term maintainability. Source code, installation instructions, and documentation are freely available at https://github.com/PharosKCL/pyslyde. Users and contributors are encouraged to submit issues, propose enhancements, and extend the toolkit to support emerging pathology and biomedical imaging analysis.

Author Contributions

G.V. developed the original PySlyde framework and initial codebase. C.E. subsequently undertook substantial architectural refactoring and extension of the software, including slide-processing and masking components, annotation and data I/O functionality, feature-extraction and model-loading infrastructure, input validation, stitching and error handling, and associated unit and integration testing. A.Baptista, H.R., E.P., M.L., A.Bhalla, F.M. and M.P. extended the functionality through annotation handling, preprocessing, stain normalisation, encoder integration, error handling testing and documentation. M.P., E.P. and C.P. developed and executed the unit tests. E.IS., C.P., J.G. and F.Z assisted with troubleshooting and software review. L.R. provided pathologist insight. G.V., A.B., C.E., H.R. wrote the manuscript, and A.G., C.B. and D.PC reviewed it.

Acknowledgements

We thank members of the King's College London Cancer Bioinformatics Team, the PharosAI Programme, and collaborators at The Alan Turing Institute, The Francis Crick Institute and Barts Life Sciences for feedback, testing, and support. Holly Rafique, Pre-Doctoral Fellow NIHR303406, is funded by the NIHR, Anita Grigoriadis, Gregory Verghese, Mengyuan Li, Elizabeth Ing-Simmons, Concetta Piazzese, Fan Zhang and Enrico Parisini are funded by the UK government, Research Ventures Catalyst Programme, Department of Science Innovation

DRAFT

and Technology, and the Guy's Cancer Charity under PharosAI for this research project. Anthony Baptista, Chima Eke and Anita Grigoriadis acknowledge support from the CRUK City of London Centre Award [CTRQQR-2021/100004]. Christopher R.S. Banerji is supported by a King's College London AI+ Fellowship. Dinis Pedro Calado acknowledges support from The Francis Crick Institute, which receives core funding from Cancer Research UK (CC2078), the UK Medical Research Council (CC2078) and the Wellcome Trust (CC2078). Dinis Pedro Calado and Anita Grigoriadis acknowledge support from the UK Medical Research Council (MR/W025221/1). Lucy Ryan is funded by the Pathological Society of Great Britain and Ireland and the Jean Shanks Foundation (JSPS CPF 1023 05). The views expressed in this publication are those of the author(s) and not necessarily those of the NIHR.

References

Anysphere. (n.d.). *Cursor*. https://cursor.com/.

Bankhead, P., Loughrey, M. B., Fernández, J. A., Dombrowski, Y., McArt, D. G., Dunne, P. D. J., McQuaid, S., Gray, R. T., Murray, L. J., Coleman, H. G., James, J. A., Salto-Tellez, M., & Hamilton, P. (2017). QuPath: Open source software for digital pathology image analysis. *Scientific Reports*, *7*. https://doi.org/10.1038/s41598-017-17204-5

Bilal, M., Aadam, Raza, M., Altherwy, Y., Alsuhaibani, A., Abduljabbar, A., Almarshad, F., Golding, P., & Rajpoot, N. (2025). *Foundation models in computational pathology: A review of challenges, opportunities, and impact*. https://doi.org/10.48550/arXiv.2502.08333

Chen, R. J., Ding, T., Lu, M. Y., Williamson, D. F. K., Jaume, G., Song, A. H., Chen, B., Zhang, A., Shao, D., Shaban, M., Williams, M., Oldenburg, L., Weishaupt, L. L., Wang, J. J., Vaidya, A., Le, L. P., Gerber, G. K., Sahai, S., Williams, W., & Mahmood, F. (2024). Towards a general-purpose foundation model for computational pathology. *Nature Medicine*, *30*, 850–862. https://doi.org/10.1038/s41591-024-02857-3

Chen, S., Parreno-Centeno, M., Booker, G., Verghese, G., Mohamed, F. S., Arslan, S., Pandya, P., Oozeer, A., D'Angelo, M., Barrow, R., Nelan, R., Sobral-Leite, M., Martino, F. de, Brisken, C., Smalley, M. J., Lips, E. H., Gillett, C., Jones, L. J., Banerji, C. R. S., … Grigoriadis, A. (2026). Normal breast tissue (NBT)-classifiers: Advancing compartment classification in normal breast histology. *Npj Breast Cancer*, *12*(1), 41. https://doi.org/10.1038/s41523-026-00896-2

Dey, S., Banerji, C. R. S., Basuchowdhuri, P., Saha, S. K., Parashar, D., & Chakraborti, T. (2025). Generating crossmodal gene expression from cancer histopathology improves multimodal AI predictions. *Nature Communications*, *17*. https://doi.org/10.1038/s41467-025-66961-9

Dolezal, J. M., Kochanny, S., Dyer, E., Ramesh, S., Srisuwananukorn, A., Sacco, M., Howard, F. M., Li, A., Mohan, P., & Pearson, A. T. (2024). Slideflow: Deep learning for digital histopathology with real-time whole-slide visualization. *BMC Bioinformatics*, *25*(1), 134. https://doi.org/10.1186/s12859-024-05758-x

Goode, A., Gilbert, B., Harkes, J., Jukic, D., & Satyanarayanan, M. (2013). OpenSlide: A vendor-neutral software foundation for digital pathology. *Journal of Pathology Informatics*, *4*(1), 27. https://doi.org/10.4103/2153-3539.119005

Janowczyk, A., Zuo, R., Gilmore, H., Feldman, M. D., & Madabhushi, A. (2019). HistoQC: An open-source quality control tool for digital pathology slides. *JCO Clinical Cancer Informatics*, *3*, 1–7. https://doi.org/10.1200/CCI.18.00157

Laak, J. A. van der, Litjens, G. J. S., & Ciompi, F. (2021). Deep learning in histopathology: The path to the clinic. *Nature Medicine*, *27*, 775–784. https://doi.org/10.1038/s41591-021-01343-4

DRAFT

Macenko, M., Niethammer, M., Marron, J. S., Borland, D., Woosley, J. T., Guan, X., Schmitt, C., & Thomas, N. E. (2009). A method for normalizing histology slides for quantitative analysis. *2009 IEEE International Symposium on Biomedical Imaging: From Nano to Macro*, 1107–1110. https://doi.org/10.1109/ISBI.2009.5193250

McGenity, C., Clarke, E. L., Jennings, C., Matthews, G., Cartlidge, C., Freduah-Agyemang, H., Stocken, D. D., & Treanor, D. (2024). Artificial intelligence in digital pathology: A systematic review and meta-analysis of diagnostic test accuracy. *Npj Digital Medicine*, *7*(1), 114. https://doi.org/10.1038/s41746-024-01106-8

Meta Platforms, Inc. (n.d.). *RocksDB: A persistent key-value store for fast storage environments*. https://rocksdb.org/.

Omar, M., Fanelli, G. N., Socciarelli, F., Ullanat, V., Puchala, S. R., Wen, J., Chowdhury, A., Valencia, I., Scatena, C., Marchionni, L., Umeton, R., & Loda, M. (2025). Antibody-based multiplex image analysis: Standard analytical workflows and artificial intelligence tools for pathologists. *Laboratory Investigation*, *105*. https://doi.org/10.1016/j.labinv.2025.104220

OpenAI. (n.d.). *Codex*. https://openai.com/codex/.

Pocock, J., Graham, S., Vu, Q. D., Jahanifar, M., Deshpande, S., Hadjigeorghiou, G., Shephard, A. J., Bashir, R. M. S., Bilal, M., Lu, W., Epstein, D. B. A., Minhas, F. A., Rajpoot, N. M., & Raza, S. E. A. (2022). TIAToolbox as an end-to-end library for advanced tissue image analytics. *Communications Medicine*, *2*. https://doi.org/10.1038/s43856-022-00186-5

Rosenthal, J., Carelli, R., Omar, M., Brundage, D., Halbert, E., Nyman, J., Hari, S. N., Allen, E. M. van, Marchionni, L., Umeton, R., & Loda, M. (2022). Building tools for machine learning and artificial intelligence in cancer research: Best practices and a case study with the PathML toolkit for computational pathology. *Molecular Cancer Research*, *20*, 202–206. https://doi.org/10.1158/1541-7786.MCR-21-0665

Saillard, C., Jenatton, R., Llinares-López, F., Mariet, Z., Cahané, D., Durand, E., & Vert, J.-P. (2024). *H-optimus-0*. https://github.com/bioptimus/releases/tree/main/models/h-optimus/v0

Schneider, C. A., Rasband, W. S., & Eliceiri, K. W. (2012). NIH image to ImageJ: 25 years of image analysis. *Nature Methods*, *9*, 671–675. https://doi.org/10.1038/nmeth.2089

Song, A. H., Jaume, G., Williamson, D. F. K., Lu, M. Y., Vaidya, A., Miller, T. R., & Mahmood, F. (2023). Artificial intelligence for digital and computational pathology. *Nature Reviews Bioengineering*, *1*, 930–949. https://doi.org/10.1038/s44222-023-00096-8

Symas Corporation. (n.d.). *LMDB: Lightning memory-mapped database*. https://www.symas.com/lmdb.

Vahadane, A., Peng, T., Sethi, A., Albarqouni, S., Wang, L., Baust, M., Steiger, K., Schlitter, A. M., Esposito, I., & Navab, N. (2016). Structure-preserving color normalization and sparse stain separation for histological images. *IEEE Transactions on Medical Imaging*, *35*, 1962–1971. https://doi.org/10.1109/TMI.2016.2529665

Verghese, G. E., Lennerz, J. K., Ruta, D., Ng, W., Thavaraj, S., Siziopikou, K. P., Naidoo, T., Rane, S. U., Salgado, R., Pinder, S. E., & Grigoriadis, A. (2023). Computational pathology in cancer diagnosis, prognosis, and prediction – present day and prospects. *The Journal of Pathology*, *260*, 551–563. https://doi.org/10.1002/path.6163

Verghese, G. E., Li, M., Liu, F., Lohan, A., Kurian, N. C., Meena, S., Gazinska, P., Shah, A., Oozeer, A., Chan, T. W. S., Opdam, M., Linn, S. C., Gillett, C., Alberts, E., Hardiman, T., Jones, S., Thavaraj, S., Jones, J. L., Salgado, R., … Grigoriadis, A. (2023). Multiscale deep learning framework captures systemic immune features in lymph nodes predictive of triple negative breast cancer outcome in large-scale studies. *The Journal of Pathology*, *260*, 376–389. https://doi.org/10.1002/path.6088

DRAFT

Zhang, A., Jaume, G., Vaidya, A., Ding, T., & Mahmood, F. (2025). Accelerating data processing and benchmarking of AI models for pathology. *arXiv Preprint arXiv:2502.06750*. https://doi.org/10.48550/arXiv.2502.06750

Zimmermann, E., Vorontsov, E., Viret, J., Casson, A., Zelechowski, M., Shaikovski, G., Tenenholtz, N., Hall, J., Fuchs, T. J., Fusi, N., Liu, S., & Severson, K. (2024). Virchow2: Scaling self-supervised mixed magnification models in pathology. *ArXiv*, *abs/2408.00738*. https://doi.org/10.48550/arXiv.2408.00738

DRAFT